17-214/514: Agentic Software Development

Software Architecture and Subsystem Design

Hammad Ahmad and Claire Le Goues • Monday, October 26, 2026 • Lecture

What did we talk about last time?

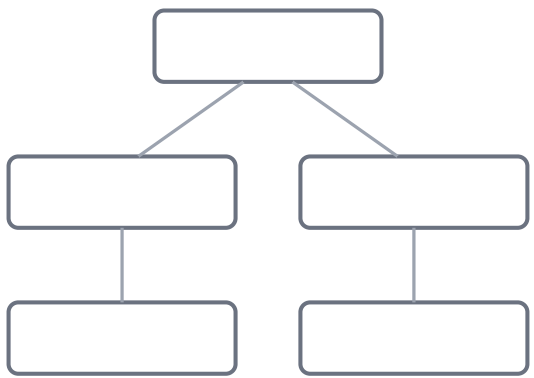
- Supervising an agent on a big task takes planning: divide the work, decide what each piece has to pass, limit what the agent can touch, and keep an independent record of what it did.
- A repo-scale change splits into slices that can be checked and reverted individually.
- A model reads and **usually** follows instructions. Permissions should be enforced by a harness.
- Agents can pass checks the wrong way (reward hacking) or follow instructions hidden in their input (prompt injection).

Learning goals for today

- Apply data, operations, and structure reasoning to subsystems
- Decide where a subsystem boundary goes, and which subsystem is responsible for each piece of data
- Explain layers, the dependency rule, ports and adapters, and Facade
- Argue **against** a new boundary when nothing calls for it yet
- Use the architecture to scope an agent's change, and ask for a check that could see the problem

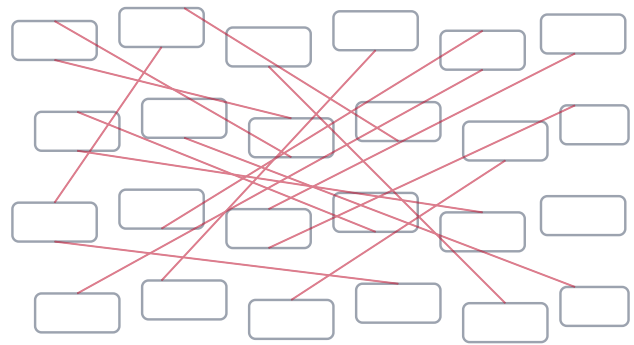
What makes a large codebase hard to change?

Five classes



One person can hold all of it

Fifty classes



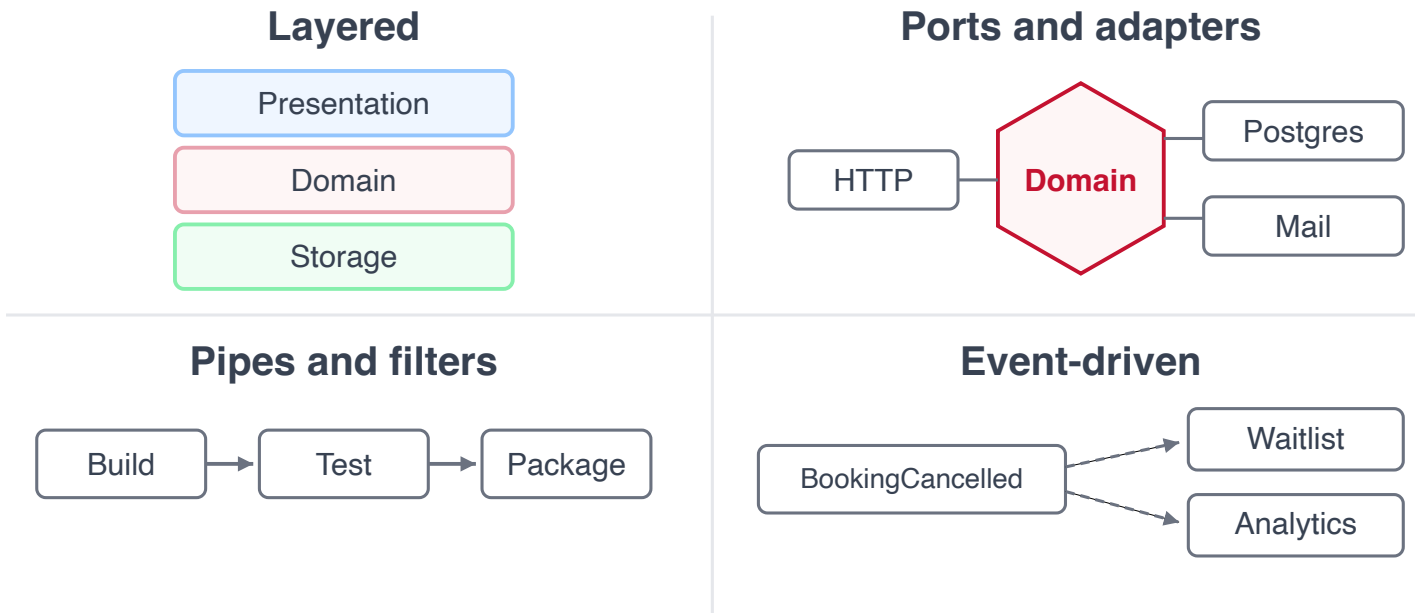
Several people and an agent change it

Design at the scale of subsystems

What is software architecture?

- System design consists of data, operations, and structure.
- **Architecture** asks about the same general categories at the level of **subsystems.**
  - A subsystem is a large part of the system with its own data and API, such as bookings, accounts, or billing.
- Different architectures offer tradeoffs in terms of **quality** attributes, rather than functionality.
- Reasonable heuristic: architectural decisions are hard to reverse later.

Architectural styles



A style is a named shape for a whole system, the way a design pattern is a named shape for a few classes. We've already built two of these (a CI pipeline and `BookingCancelled`).

Pipes and filters: data flows through a chain of stages

What it is

- Each stage (a filter) takes the previous stage's output and passes its own output along.
- Stages don't call each other. A CI pipeline (build, test, package) works this way.

When to use it

- It fits work that transforms data step by step, like builds, compilers, and data processing.
- You can add, swap, or reorder a stage, and test each one alone.
- It fits badly when stages need shared state, or when a user waits on a back-and-forth.

Event-driven: parts announce events and others react

What it is

- A part announces that something happened, and other parts listen and react.
- The sender doesn't know who is listening. When a booking is cancelled, bookings announces `BookingCancelled`, and the waitlist and analytics react.

When to use it

- It fits when several parts react to one change, and the sender shouldn't wait for them.
- You can add a listener without touching the sender.
- The flow is harder to follow, since no one place shows what happens after an event. Ordering and lost events need thought.

Layered architecture

Layered architecture

What it is

- Code is split into ordered layers that each depends on the one "below" it.
- Example: Web services often use presentation, domain, and storage (`api`, `domain`, `storage` in the booking system).
- Each layer handles one concern (e.g., overlap rule is in the domain; storage just saves bookings).

When/how to use it

- It fits when the work divides into levels.
- You can change or test one layer at a time.
- A small change often means editing every layer.

The dependency rule: every dependency points toward the domain

Plain layering

```
// domain
class BookingService {
  PostgresBookingStore store;
}
```

Testing the domain needs a database, and switching databases means editing the domain.

With the rule

```
// domain
interface BookingStore { void save(Booking b); }
class BookingService { BookingStore store; }
// storage
class PostgresBookingStore
    implements BookingStore { ... }
```

Switching databases means writing a new `BookingStore` implementation. The domain doesn't change.

Why? The domain changes least often, so everything else depends on it. Meanwhile, When the program runs, the domain still calls storage. In the code, storage names the domain's interface.

Enforcing layers with an architecture test

```
@ArchTest
static final ArchRule apiNeverTouchesStorage =
    noClasses().that().resideInAPackage("..api..")
        .should().dependOnClassesThat().resideInAPackage("..storage..");

@ArchTest
static final ArchRule domainNeverTouchesStorage =
    noClasses().that().resideInAPackage("..domain..")
        .should().dependOnClassesThat().resideInAPackage("..storage..");
```

- (We've seen these before.)
- An architecture test enforces the rule as a build failure (ArchUnit for Java, dependency-cruiser for TypeScript, import-linter for Python).

Is this architecture test useful?

- **What claim does it check?** Nothing in `domain` refers to anything in `storage`. ("The architecture looks fine" isn't a claim.)
- **Can it fail?** We can check, by running it on the left-hand `BookingService` from the dependency-rule slide.
- **Could it see the problem?** Any domain class could use storage directly, so unit tests of one class won't find it.

(We can ask the same three questions of any verification-oriented check! Could they "see" the problem they're trying to catch?)

Adding architecture tests to an existing codebase

- Apache Flink added ArchUnit rules in 2021 to an existing codebase.
- Each rule with existing violations was "frozen", recording the violations, so only new violations count as "failures." This is good practice for introducing new QA to existing projects.
- One rule started out with about 700 recorded violations (so, plenty).

> If there are existing violations which cannot be fixed right away, the rule must be *frozen* by wrapping it in `FreezingArchRule.freeze()`. This will add the rule to the violation store that records the existing violations. Add the new stored violations file within `violations` to your commit.

Apache Flink

When the diagram and the code disagree

- Diagrams and code often do not change together.
- Chromium's design doc still describes a "WebKit" layer and a "Glue" layer. Chromium replaced WebKit with its own fork, Blink, in 2013.
- One way to check a diagram is to regenerate the picture from the code and compare.

How Chromium Displays Web Pages

WebKit: Rendering engine shared between Safari, Chromium, and all other WebKit-based browsers. The **Port** is a part of WebKit that integrates with platform dependent system services such as resource loading and graphics.

Glue: Converts WebKit types to Chromium types. This is our "WebKit embedding layer." It is the basis of two browsers, Chromium, and test_shell (which allows us to test WebKit).

The Chromium Projects

Checking a diagram against the code

Running `jdeps` (it ships with the JDK) on a small example project, where one class in `api` uses `storage` directly:

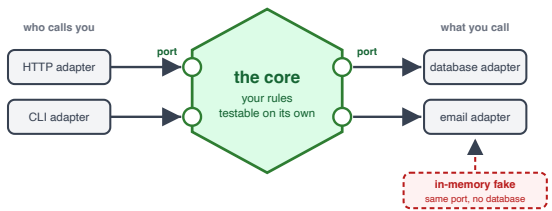
```
$ jdeps -verbose:package out | grep -- '-> booking' | awk '{print $1, $2, $3}'
booking.api -> booking.domain
booking.api -> booking.storage
booking.storage -> booking.domain
```

- The layering says `api` reaches storage only through the domain. The second line says otherwise.
- An agent can run a tool like this (dependency-cruiser for TypeScript, pydeps for Python) and draw the result for you.
- Each edge the diagram doesn't show means either the code needs fixing or the diagram does.

Ports and adapters

Ports and adapters: domain at center of dependencies

What it is

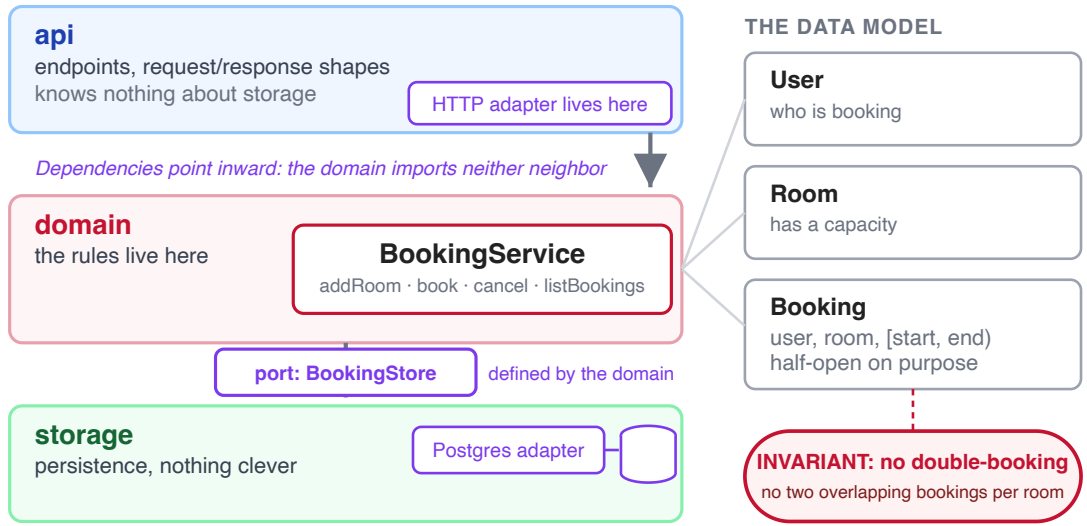


- For each outside system it uses (storage, mail, the clock), the domain defines a **port** as an interface. An **adapter** implements it for one technology.
- Every dependency points at the domain. There's no "above" or "below."

When to use it

- It fits when the domain logic should be mostly fixed but outside systems might change.
- Any outside system can be replaced without touching the domain. Useful for testing and for swapping out technology.
- Each outside system adds an interface and an adapter, which can be unecessary complexity depending on the system.

The booking system is easy adapt to use ports



- The `BookingStore` port and the adapters at each edge are new (in violet).
- We have built each piece before (the injected `Clock`, the `RecordingNotifier` test double, the `MailgunNotifier` adapter).

Ports and adapters help decide where cross-cutting code goes

Logging, auth, and validation are needed almost everywhere (they're "cross-cutting"), so they tend to creep into the domain:

```
// domain layer
public Booking book(String sessionToken, Slot slot, int price) {
  log.info("book " + slot + " by " + sessions.userFor(sessionToken));
  ...
}
```

- Code that depends on how a request arrives goes in an adapter. Rules that apply regardless go into the domain.
- So the HTTP adapter turns the session token into a `User`, and `book` takes the `User`.
- The adapter checks that a date is real. "No overlapping bookings" stays in the domain.
- Logging is split. Request logs go in adapters, and domain events can go through a port.

Boundaries and ownership

A subsystem can be built as a module or as a service

- **As a module**, it is code in the same program. Other parts call it with method calls, and it is deployed with everything else.
- **As a service**, it is its own running program. Other parts call it over the network, and it is deployed on its own.
  - Layers and ports and adapters work the same either way.
- The rest of this section is about when a subsystem should be its own service.

When is it worth making a subsystem its own service?

- A separate service adds an API, its own deployment, and network calls between it and everything else.
- It's usually worth it when two parts differ in how they change, release, or are staffed:

Change

Parts that change for different reasons can be separated. If most features touch both, keep them together.

Release

Different release schedules are a reason to separate.

Team

Each subsystem has one team, or one person, responsible for it.

Splitting into separate services turns method calls into network calls

Before each booking, bookings looks up the student in accounts.

Accounts is a module in the same program
Bookings calls a method on accounts. The call is fast, and it can't fail unless the whole program has. A problem shows up in one stack trace.

Accounts is a separate service
Bookings sends an HTTP request to accounts. The request is slower, and it can fail on its own if accounts is slow or down. A problem is spread across two programs and two logs.

If accounts is down, nobody can book. Each split adds calls like this one, which is part of what a separate service costs.

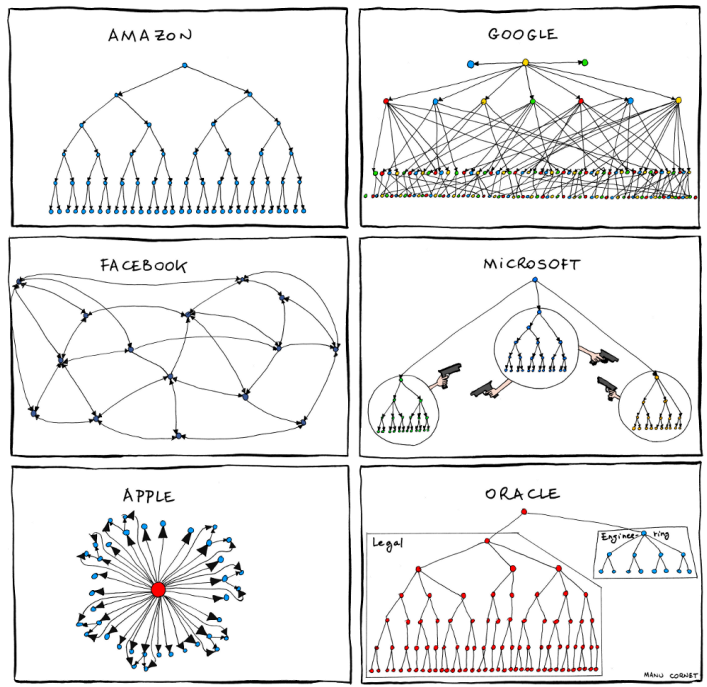
Conway's Law

> The basic thesis of this article is that organizations which design systems (in the broad sense used here) are constrained to produce designs which are copies of the communication structures of these organizations. We have seen that this fact has important implications for the management of system design. Primarily, we have found a criterion for the structuring of design organizations: a design effort should be organized according to the need for communication.

> *Examples*. A contract research organization had eight people who were to produce a COBOL and an ALGOL compiler. After some initial estimates of difficulty and time, five people were assigned to the COBOL job and three to the ALGOL job. The resulting COBOL compiler ran in five phases, the ALG0L compiler ran in three.

Melvin Conway, Datamation (1968)

Used in the other direction, it suggests drawing boundaries that a team can be responsible for.



Manu Cornet

How separate should notifications and billing be from bookings?

The booking platform is adding notifications and billing.



Inside bookings
Part of the bookings code. This is the cheapest. It fits when the feature shares bookings' rules.

Its own module
A module next to bookings, in the same program. This is usually the default.

Its own service
A separate program. This is worth it when a team, a release schedule, or an audit calls for it.

<- cheaper to build and change

more to build and run ->

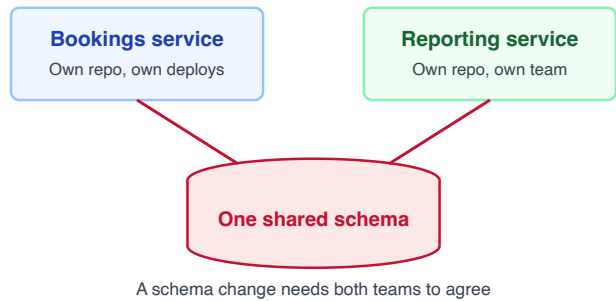
- Notifications could be its own module, one that listens for booking events.
- Billing could be its own module for now, with a boundary clean enough to extract later if warranted. This kind of decision should be documented (ADR).

API between subsystems/API design rehash

- At a subsystem boundary, the API's users are the other subsystems and their maintainers. They depend on it the same way outside users depend on a library.
- So: API design applies! An API that is hard to misuse prevents mistakes in the calling subsystem, additive changes avoid breaking it. Hyrum's Law remains relevant.
- Contract tests can check that what the caller expects still matches what the provider does.

Only one subsystem should write any one kind of data

- One subsystem writes each piece of data and enforces the rules about it. Everyone else asks that subsystem, or listens for its events.
- A shared table breaks this. A service's tables are like a class's private fields, and here reporting reads bookings' tables directly. If bookings renames a column, reporting breaks.
- Instead, reporting can call bookings' API, listen for bookings' events and keep its own copy, or read an export that bookings publishes and keeps stable.



Which subsystem should count active bookings and enforce the booking limit?

A new rule: each student has a tier, undergrad or grad. **Undergrads can hold at most three active bookings, and grad students five.**

- Accounts? It knows each student's tier, and the limit depends on the tier.
- Bookings? Every booking and cancellation changes the count, and the check happens at booking time.



The limit is a rule about bookings, so it should go in bookings.

Better replies to "Shall I proceed?"

Agent plans usually end with "Shall I proceed?" There are more useful answers than "yes":

- **Ask for the check, first.** "Write the tests every `BookingStore` has to pass before you write the adapter, and show me them failing."
- **Challenge one item.** "Why is pricing its own service? What would it cost as a module?"
- **Ask what's missing.** "Which data does each part own, and does anything else write to it?"
- **Ask for the evidence.** "You said the architecture tests pass. Which rule do they check, and have you seen one fail?"

Let's practice: An agent's proposal for the booking platform, which a team of four builds as one deployable today:

```
Services, each with its own database:
1. bookings       bookings and rooms
2. pricing        computes a price for a slot
3. limits         each student's active-booking count;
                  bookings calls it to add one
                  before every booking
4. notifications  listens for booking events
5. accounts       students, tiers, login
6. analytics      reads the bookings database directly
                  for the "busiest rooms" dashboard

Verification: each service ships with unit tests;
all pass.

Shall I proceed?
```

Which services would you keep separate? Then, with a neighbor, write the reply you would send.

Canvas → Quizzes → Lecture 15 In-Class Quiz

Problems with the agent's plan

- At four people, most of these can be modules in one deployable (pricing, notifications, analytics, and probably bookings with limits). Accounts is a reasonable service if login belongs to a different team.
- Analytics reads another service's database, which makes it a shared database.
- Limits adds one per booking, and nothing ever subtracts one, so after the first cancellation the count is wrong. The rule lives away from the bookings it counts.
- The plan's verification fails the third question about checks. Each service's unit tests stay inside that service, so none of them could see the count go wrong between bookings and limits.
- A reply: "What updates the count when a booking is cancelled? Before you build anything, write a test across bookings and limits that fails today, and an architecture test that nothing outside bookings reads its tables."

One service or many?

Microservices: many small services, each deployed on its own

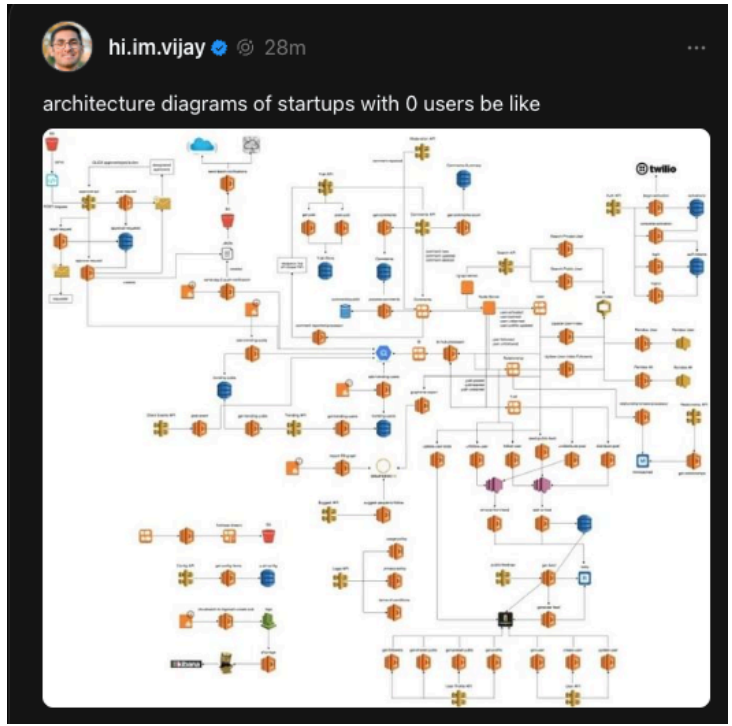
What it is

- The system is split into small services, each deployed separately with its own data.
- Services talk over the network, through APIs or events.

When to use it

- It fits when parts need to release, scale, or be staffed independently, often across many teams.
- Each service can change and deploy on its own schedule.
- Every boundary costs network calls, a deploy pipeline, and cross-service debugging. Splitting before anything calls for it is speculative generality at system scale.

Microservices, and splitting too early



ProgrammerHumor.io

Segment: from 140 services back to one

Goodbye Microservices: From 100s of problem children to 1 superstar

- Segment ran a core part of its product as more than 140 small services, maintained by a small team.
- Eventually they merged them back into a single service.

Twilio Segment

> In the early days of Twilio Segment, we reached a tipping point with a core piece of Twilio Segment's product. It seemed as if we were falling from the microservices tree, hitting every branch on the way down. Instead of enabling us to move faster, the small team found themselves mired in exploding complexity. Essential benefits of this architecture became burdens. As our velocity plummeted, our defect rate exploded.

Modular monolith: one deployable with clear internal boundaries

What it is

- One codebase and one deployable, split into modules whose boundaries are enforced, often by architecture tests.
- The kind of monolith to avoid has no internal boundaries (a "big ball of mud").

When to use it

- It fits most systems built by one team or a few, and is often a good default.
- One deployment reduces distributed system-based headaches.
- Everything ships and scales together. Because there **are** boundaries, extracting a service later is mostly refactoring.

Modular monoliths

Deconstructing the Monolith: Designing Software that Maximizes Developer Productivity

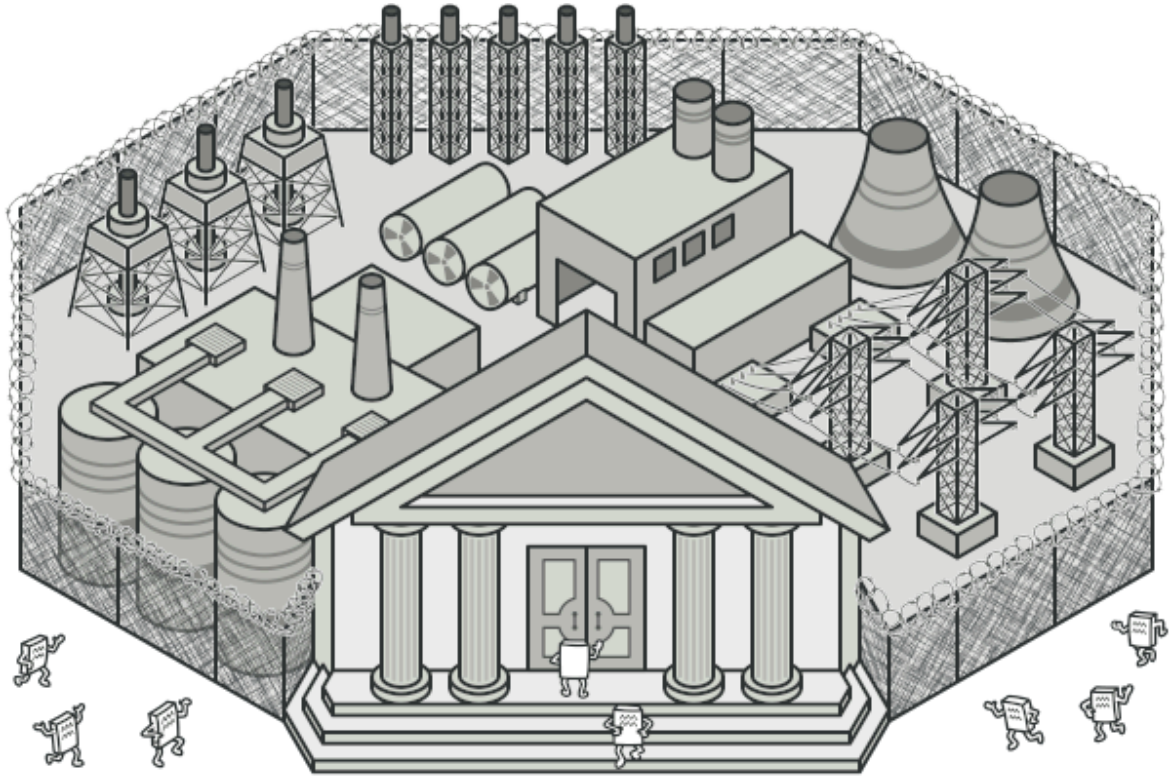
Microservices surged in popularity in recent years and were touted as the end-all solution to all of the problems arising from monoliths. Yet our own collective experience told us that there is no one size fits all best solution, and microservices would bring their own set of challenges. We chose to evolve Shopify into a modular monolith, meaning that we would keep all of the code in one codebase, but ensure that boundaries were defined and respected between different components.

Shopify Engineering

What a modular monolith looks like

```
reservation-platform/          one repo, one deployable
  bookings/                    module: bookings and its data
    api/       <- the only package other modules may import
    internal/  <- private, enforced by an architecture test
  pricing/
    api/
    internal/
  notifications/               listens for bookings' events,
                               never imports bookings.internal
```

- Each module has a public `api` package, an `internal` package, its own data, and its own tests.
- Other modules go through `api` or listen for events.
- Extracting a module into a service later mostly means turning `api` into a network API.



Refactoring.Guru

Facade

The problem: The booking handler calls three parts of the system, so it has to know how they fit together.

```
// before: the handler must follow the whole sequence
Slot slot = rooms.slotFor(roomId, start, end);
int price = pricing.quote(user, slot);
Booking booking = bookings.book(user, slot, price);
```

```
// after: one entry point in front of those parts
Booking booking = reservations.book(user, roomId, start, end);
```

`Reservations` is a **Facade**, one simple entry point in front of several parts. The handler no longer needs to track the sequence.

Facade trade-offs

- It can grow into a God class, with every new feature adding one more method to `Reservations`.
- A caller that needs one part directly can still call it. The Facade is the default way, not the only way.
- A module's public `api` package is often a Facade over the module's internals.

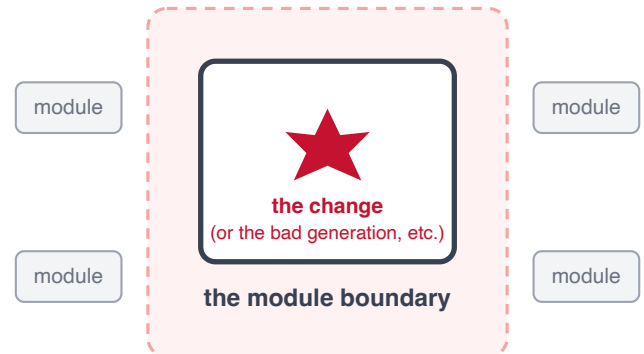
Running a monolith as several copies

- A modular monolith can still run as several copies behind a proxy, with each copy serving some of the requests.
- The catch is that anything one copy keeps in memory, the others can't see. A session map is one example. A count of each student's active bookings is another.
- Where that state should live instead is Wednesday's topic.

Architecture and agents

Using boundaries to scope an agent's task

- A boundary limits what a change inside it can reach, for subsystems as for modules.
- So a well-bounded subsystem is a natural unit of work for an agent, and a reviewer knows what should have changed.
- "Implement a Redis adapter behind `BookingStore`" is bounded by the port. "Improve performance across the platform" could touch anything.
- Choosing the subsystems, layers, and ports is expensive to reverse, which is a reason to make that call yourself.



a boundary decides (among other things)
how far wrong can spread

Module rules in agent instruction files

Rules about one module can load only when the agent works on that module (they're still read, not necessarily obeyed). In Claude Code, `.claude/rules/domain.md`:

```
---
paths:
  - "src/main/java/booking/domain/**"
---
The domain imports nothing from booking.api or booking.storage.
Storage is reached only through the BookingStore port.
```

- This file loads when Claude Code reads a file under `domain/`.
- Codex reads one `AGENTS.md` per directory, from the repo root down to the directory it starts in.

Reviewing a large diff, starting with imports

- In a large generated diff, boundary violations are usually quicker to find than logic errors, so they are a reasonable place to start.
- Most show up in the import list, such as a module importing another module's internals, or a domain file importing an adapter's types. Writes to another subsystem's data are harder to spot, since they can go through a shared table.
- An agent can produce this list on request ("list every import this diff adds that crosses a module").
- Architecture tests already catch the violations they have rules for, so review mostly finds the ones no rule covers yet.

A4: due Wednesday

- Due Wednesday at 11:59pm: `PROPOSAL.md`, your characterization tests, and the refactor, with `planning/` and `transcripts/`.
- `ARCHAEOLOGY.md` is graded again as it stands at the deadline.
- Submit the link to your final commit (not the repository) on Canvas.

Summary

- Architecture is design for subsystems, where decisions are expensive to reverse.
- Layers, the dependency rule, and ports and adapters keep the domain independent of storage, HTTP, and vendors. Architecture tests enforce them.
- A boundary is worth it when parts change, release, or are staffed differently, and each piece of data should have one owner.
- A modular monolith is often a good default. Splitting early means paying for network calls and cross-service debugging up front.
- Boundaries give an agent's task its scope. For any check, ask what claim it checks, whether you've seen it fail, and whether it could see the problem.

What's next

W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14
A1 out		A1 due		A2 due · M1		A3 due	A4 chkpt	A4 due	M2		A5 due		A6 due

- **Wednesday:** State at scale. Where state lives when the system runs as copies, Docker Compose, and measuring whether a design scales. A4 is due at 11:59pm.
- **Friday, Lab 9:** Run a system we provide as copies, and measure what breaks.
- **Next week:** Concurrency on Monday, then Midterm 2 on Wednesday.